

RAG Architecture Patterns

A Comprehensive Technical Reference for Production Systems

This reference covers retrieval-augmented generation (RAG) from first principles through advanced production patterns — including chunking strategies, embedding models, retrieval methods, pipeline architecture, evaluation frameworks, and operational best practices.

DOCUMENT TYPE
Technical Reference

SCOPE
Production RAG Systems

VERSION
2025 Edition

Table of Contents

1	Introduction to RAG and the Retrieve-Then-Generate Paradigm	3
2	Document Chunking Strategies	3
3	Embedding Models	4
4	Retrieval Methods	5
5	Advanced RAG Patterns	6
6	RAG Pipeline Architecture	7
7	Evaluation Metrics and Frameworks	8
8	Common Pitfalls and Best Practices	9

1 Introduction to RAG and the Retrieve-Then-Generate Paradigm

Retrieval-Augmented Generation (RAG) grounds large language model (LLM) outputs in verifiable, up-to-date external knowledge, dramatically reducing hallucination rates and enabling domain-specific applications without full fine-tuning.

1.1 What Is RAG?

RAG is an architectural pattern that augments an LLM's parametric knowledge with non-parametric, retrieved context at inference time. First formalised by Lewis et al. (2020, Facebook AI Research), the core idea is simple: before generating a response, retrieve the k most relevant documents from an external corpus and inject them into the model's context window.

This addresses two fundamental limitations of standalone LLMs: (1) knowledge cutoff — models cannot access information published after training; (2) hallucination — models sometimes generate confident but factually incorrect outputs when relying solely on parametric memory.

1.2 The Retrieve-Then-Generate Paradigm

The canonical RAG pipeline consists of three stages executed at query time:

1. **Retrieval:** Given a user query q , compute a similarity score between an embedded representation of q and all document chunk embeddings in a vector store. Return the top- k chunks (typically $k = 3-10$).
2. **Augmentation:** Construct a prompt that includes the retrieved context chunks alongside the original query, using a structured template (e.g., Context: {docs} \n\n Question: {q} \n Answer:).
3. **Generation:** Pass the augmented prompt to the LLM, which synthesises an answer grounded in the provided context.

1.3 RAG vs. Fine-Tuning

Dimension	RAG	Fine-Tuning
Knowledge update	Re-index corpus (minutes–hours)	Full retraining (days–weeks)
Traceability	High — source chunks cited	Low — knowledge opaque
Cost (ongoing)	Retrieval latency + storage	Retraining compute
Hallucination risk	Low (grounded in context)	Moderate
Domain adaptation	Corpus-driven	Data-driven

Industry Benchmark: RAG reduces hallucination rates by 40–60% compared to vanilla LLM generation on knowledge-intensive QA tasks (Gao et al., 2023). On TriviaQA, RAG with DPR achieves 56.8% exact match vs. GPT-3's 29.9% zero-shot baseline.

1.4 Naive vs. Advanced RAG

Naive RAG applies fixed-size chunking, a single dense retriever, and direct stuffing of top- k chunks into the prompt. While functional, it suffers from retrieval precision issues, context window misuse, and sensitivity to chunk quality. **Advanced RAG** introduces query transformation, hybrid retrieval, re-ranking, contextual compression, and structured pipeline orchestration to address these gaps.

2 Document Chunking Strategies

Chunking is the most impactful — and most under-appreciated — parameter in RAG system design. The chunk is the atomic unit of retrieval; its size and boundaries determine what context the model sees.

2.1 Fixed-Size Chunking

Split documents into chunks of exactly N characters (or tokens), with an optional overlap of O characters to preserve context across boundaries.

✓ PROS

- Deterministic and fast — $O(n)$ time
- Predictable embedding sizes
- Easy to implement and reason about

✗ CONS

- Splits mid-sentence or mid-concept
- Ignores semantic structure entirely
- Requires careful overlap tuning

Recommended sizes: 256–512 tokens for dense QA tasks; 512–1024 tokens for document summarisation. Overlap: 10–15% of chunk size (e.g., 50 tokens for 512-token chunks). Benchmark studies show 512-token chunks with 50-token overlap maximise retrieval recall on most English corpora.

2.2 Sentence-Based Chunking

Use a sentence boundary detector (e.g., SpaCy, NLTK Punkt) to split text at natural sentence endings, then group consecutive sentences into chunks up to a maximum token budget.

✓ PROS

- Preserves grammatical completeness
- Better semantic coherence per chunk
- Suitable for factoid retrieval

✗ CONS

- Variable chunk sizes complicate batching
- Sentence detector errors propagate
- Misses multi-sentence concepts

Recommended sizes: 3–7 sentences per chunk; target ≤ 384 tokens. Particularly effective for legal and scientific corpora where sentence-level precision matters.

2.3 Semantic Chunking

Embed every sentence independently, then compute cosine similarity between adjacent sentence embeddings. Insert a chunk boundary wherever similarity drops below a threshold (typically 0.7–0.8), signalling a topic transition.

✓ PROS

- Chunks align with topical units
- Highest semantic coherence
- Improves retrieval precision by ~12%

✗ CONS

- Expensive — requires N sentence embeddings
- Threshold requires domain calibration
- Inconsistent chunk sizes

Recommended approach: Use `text-embedding-3-small` for sentence embeddings during indexing. Set threshold at 0.75. Merge chunks smaller than 128 tokens with the adjacent chunk.

2.4 Recursive Character Text Splitting

LangChain's `RecursiveCharacterTextSplitter` uses a priority-ordered list of separators — `["\n\n", "\n", ". ", ",", ""]` — attempting larger structural splits first, recursively falling back to finer-grained splits only when necessary.

✓ PROS

- Respects paragraph and sentence structure by default
- More robust than naive fixed-size
- Configurable separator hierarchy

✗ CONS

- Still character-based, not semantic
- Markdown/code requires custom separators
- No awareness of document structure

Recommended configuration: `chunk_size=512`, `chunk_overlap=64`. For Markdown, add `["## ", "### ", "\n\n"]` as leading separators. This is the recommended default for general-purpose RAG.

2.5 Chunking Strategy Comparison

Strategy	Retrieval Precision	Index Speed	Chunk Consistency	Best For
Fixed-size	Moderate	Very fast	High	Rapid prototyping
Sentence-based	Good	Fast	Moderate	Factoid QA
Semantic	High (+12%)	Slow (2–5×)	Low	Long-form docs
Recursive char	Good	Fast	Moderate	General purpose

3 Embedding Models

The embedding model maps both queries and document chunks to a shared vector space. Model selection determines the quality ceiling of retrieval — no amount of downstream re-ranking can recover from a weak embedding space.

3.1 OpenAI text-embedding-ada-002 and text-embedding-3 Series

OpenAI's embedding API remains the most widely deployed option in production RAG systems.

- **text-embedding-ada-002:** 1536-dimensional vectors; 8191-token context window; cost: \$0.0001 / 1K tokens. Strong MTEB score of 61.0. The safe default for most production systems through 2024.
- **text-embedding-3-small:** 1536 dimensions (reducible via MRL to 512 or 256); MTEB score 62.3; cost: \$0.00002 / 1K tokens — 5× cheaper than ada-002 with higher accuracy.
- **text-embedding-3-large:** 3072 dimensions; MTEB score 64.6; cost: \$0.00013 / 1K tokens. Recommended for high-precision retrieval tasks.

Matryoshka Representation Learning (MRL): text-embedding-3 models support dimension truncation without retraining. Truncating from 1536 → 256 dimensions reduces storage by 83% with only a 3–5% precision drop, enabling cost-effective large-scale deployments.

3.2 Cohere Embed

Cohere's embedding models offer a compelling open-weights alternative:

- **embed-english-v3.0:** 1024 dimensions; int8 quantisation supported natively; 512-token context; MTEB score 64.5. Supports binary embeddings (1-bit quantisation) for ultra-fast approximate search at ~97% recall.
- **embed-multilingual-v3.0:** 108 languages; 1024 dimensions; MTEB Multilingual score 62.4.

Key differentiator: Cohere's API accepts both `search_document` and `search_query` input types, applying asymmetric embeddings that improve retrieval recall by 5–8% over symmetric embeddings for short-query / long-document retrieval scenarios.

3.3 Sentence-Transformers (Open-Source)

The `sentence-transformers` library provides 100+ pre-trained models suitable for self-hosted, privacy-sensitive, or cost-constrained deployments:

- **all-mpnet-base-v2**: 768 dimensions; highest quality for English; MTEB score 57.8; 420 MB model size.
- **all-MiniLM-L6-v2**: 384 dimensions; excellent speed/quality tradeoff; 5× faster than mpnet; MTEB score 56.3; 80 MB. Ideal for low-latency requirements.
- **bge-large-en-v1.5 (BAAI)**: 1024 dimensions; MTEB score 63.6; surpasses ada-002 on English benchmarks at zero API cost.
- **e5-large-v2 (Microsoft)**: 1024 dimensions; MTEB 62.3; prepend `"query: "` and `"passage: "` prefixes for best results.

3.4 Dimensionality and Storage Considerations

Model	Dimensions	MTEB Score	Cost / 1M tokens	Storage (1M vectors)
text-embedding-3-small	1536	62.3	\$0.02	~6 GB (float32)
text-embedding-3-large	3072	64.6	\$0.13	~12 GB (float32)
cohere embed-v3	1024	64.5	\$0.10	~4 GB (float32)
bge-large-en-v1.5	1024	63.6	\$0 (self-host)	~4 GB (float32)
all-MiniLM-L6-v2	384	56.3	\$0 (self-host)	~1.5 GB (float32)

Storage calculated as: $\text{dimensions} \times 4 \text{ bytes} \times \text{num_vectors}$. int8 quantisation reduces storage by 4×. Binary embeddings reduce by 32×.

3.5 Embedding Best Practices

- **Use the same model** for indexing and query time — cross-model vector spaces are incompatible.
- **Normalise embeddings** to unit length before storing; inner product then equals cosine similarity, enabling faster SIMD operations.
- **Batch encode** during indexing: most APIs accept batches of 512–2048 texts; this reduces latency by 10–20× vs. individual requests.
- **Cache query embeddings** for repeated queries; embed queries asynchronously in high-throughput systems.

4 Retrieval Methods

No single retrieval strategy dominates across all domains. Production systems typically combine dense and sparse retrieval with downstream re-ranking to balance recall, precision, and latency.

4.1 Dense Retrieval (ANN Search)

Dense retrieval uses approximate nearest neighbour (ANN) search over the embedding vector space. Query and document embeddings are compared using cosine similarity or inner product.

- **HNSW (Hierarchical Navigable Small World)**: De-facto standard; $O(\log N)$ query time; 95–99% recall@10 at 1ms latency for 1M vectors. Implemented in FAISS, Qdrant, Weaviate, Chroma.
- **IVF-PQ (Inverted File + Product Quantisation)**: Memory-efficient for billion-scale; 64–256 bytes per vector vs. 4KB for float32 1024-dim vectors.
- **Flat (brute-force)**: 100% recall; practical only for <100K vectors; $O(N)$ per query.

FAISS benchmark (1M 128-dim vectors, HNSW M=32): 0.8ms query latency, 97.3% recall@10, 512 MB RAM. Builds in ~45 seconds.

4.2 Sparse Retrieval — BM25

BM25 (Best Match 25) is a probabilistic bag-of-words ranking function that scores document relevance based on term frequency (TF), inverse document frequency (IDF), and document length normalisation:

$$\text{Score}(D, Q) = \sum \text{IDF}(q_i) \cdot [\text{TF}(q_i, D) \cdot (k_1 + 1)] / [\text{TF}(q_i, D) + k_1 \cdot (1 - b + b \cdot |D| / \text{avgdl})]$$

Where $k_1 = 1.5$ controls TF saturation and $b = 0.75$ controls length normalisation. BM25 excels at exact-match keyword queries and technical terminology not well-represented in embedding spaces (model names, product codes, acronyms).

Implementation: Elasticsearch, OpenSearch, BM25Okapi (Python), Whoosh. Elasticsearch achieves <10ms latency on 10M documents at 99th percentile.

4.3 Hybrid Search

Hybrid search combines dense and sparse scores via Reciprocal Rank Fusion (RRF) or weighted linear combination:

$\text{RRF_score}(d) = \sum 1 / (k + \text{rank_i}(d))$ where $k = 60$ (empirically optimal)

Linear: $\text{score}(d) = \alpha \cdot \text{dense_score}(d) + (1 - \alpha) \cdot \text{sparse_score}(d)$ $\alpha \in [0, 1]$

RRF is preferred when scores are not on the same scale. A typical starting value for linear fusion is $\alpha = 0.7$ (dense-heavy). In benchmarks on MS MARCO, hybrid search improves NDCG@10 by +8–14% over dense-only retrieval.

Method	Recall@10	Precision@5	Latency (1M docs)	Exact-match strength
Dense only (HNSW)	78–85%	62–70%	1–3ms	Low
BM25 only	65–74%	55–65%	5–10ms	High
Hybrid (RRF)	84–91%	68–76%	8–15ms	High

4.4 Re-Ranking with Cross-Encoders

Bi-encoder retrieval (dense/sparse) is fast but imprecise — it scores query-document pairs independently. Cross-encoders see both query and document jointly, enabling deep attention-based relevance scoring at the cost of $O(N)$ forward passes.

Workflow: Retrieve top-100 candidates via fast ANN/BM25, then re-rank with a cross-encoder to produce a refined top-10. Common models:

- `cross-encoder/ms-marco-MiniLM-L-6-v2`: 22M params; 40ms for 100 pairs; NDCG@10 improvement: +6–12% over bi-encoder retrieval.
- `cohere rerank-english-v3.0`: API-based; state-of-the-art BEIR scores; latency ~200ms per request (network-dependent).
- `BAAI/bge-reranker-large`: 335M params; best open-source option; BEIR NDCG@10 of 54.2.

4.5 Maximal Marginal Relevance (MMR)

MMR balances relevance and diversity in the retrieved set, preventing near-duplicate chunks from dominating the context window:

$\text{MMR} = \text{argmax}_d [\lambda \cdot \text{sim}(q, d) - (1 - \lambda) \cdot \max_{\{d' \in S\}} \text{sim}(d, d')]$

Where S is the set of already-selected documents, λ controls the relevance/diversity tradeoff ($\lambda = 0.5$ is a common default). MMR is particularly valuable for multi-faceted queries where diverse perspectives improve answer quality.

5 Advanced RAG Patterns

Advanced patterns address the failure modes of naive RAG: poor retrieval for ambiguous or complex queries, context window misuse, and answer unfaithfulness.

5.1 Query Transformation

The user's original query is often poorly suited for retrieval — it may be too short, ambiguous, or presuppose knowledge. Query transformation rewrites the query before retrieval.

- **Step-Back Prompting:** Ask the LLM to generate a more abstract, principle-level question. E.g., "What GPU is in the RTX 4090?" → "What are NVIDIA's current-generation GPU architectures?" Improves retrieval by 15% on knowledge-intensive tasks.
- **Query Decomposition:** Break complex multi-hop questions into sub-queries. Execute each sub-query independently and merge retrieved contexts before generation.
- **Query Expansion:** Append synonyms, related terms, or alternative phrasings using an LLM or lexical resource (WordNet) before embedding.

5.2 HyDE — Hypothetical Document Embeddings

HyDE (Gao et al., 2022) inverts the retrieval problem: instead of embedding the short query and searching for similar long documents, use an LLM to generate a hypothetical answer document, then embed that document for retrieval.

`q_hypothetical = LLM("Write a passage that answers: " + q)`
`retrieved_docs = ANN_search(embed(q_hypothetical), corpus)`

- **Mechanism:** The hypothetical document occupies the same part of the embedding space as real answer-containing documents, narrowing the distribution mismatch between short queries and long passages.
- **Performance:** +12–18% NDCG@10 over baseline dense retrieval on TREC DL19/DL20 benchmarks.
- **Caution:** If the LLM hallucinates in the hypothetical, retrieval quality degrades. Apply only when a capable LLM (GPT-4 class) is available for hypothesis generation.

5.3 Multi-Query Retrieval

Generate N diverse query reformulations using an LLM ($N = 3\text{--}5$ is typical), retrieve top- k chunks for each, then deduplicate and union the result sets before re-ranking.

- Increases recall by 20–30% for ambiguous queries by covering multiple interpretations.

- Latency cost: $N \times \text{retrieval latency}$ (parallelisable).
- Deduplication via chunk ID set union; re-rank the merged $\sim(N \cdot k)$ candidates using a cross-encoder.

5.4 Parent-Child Chunking

Index small child chunks (128–256 tokens) for high-precision retrieval, but return their larger parent chunks (512–2048 tokens) for richer generation context.

- **Indexing:** Store parent chunks in a document store; index child chunks in the vector store with a `parent_id` pointer.
- **Retrieval:** Retrieve top- k child chunks, look up their parent chunks, deduplicate, and pass parents to the LLM.
- **Benefit:** Child chunk precision combined with parent chunk context. Particularly effective for long legal/scientific documents where a single sentence answers the query but the surrounding paragraph provides necessary context.
- **LangChain implementation:** `ParentDocumentRetriever` with `InMemoryStore` for the parent docstore.

5.5 Contextual Compression

Rather than returning entire retrieved chunks, extract only the sentences or spans within each chunk that are relevant to the query. This maximises the information density of the context window.

- **LLM Extractor:** Prompt the LLM to extract relevant sentences from each chunk. High quality but adds $\sim 100\text{--}200\text{ms}$ latency per chunk.
- **Embeddings Filter:** Re-embed each sentence in the chunk, retain only those with cosine similarity $>$ threshold (0.75) to the query embedding. Faster ($\sim 10\text{ms}$) but less nuanced.
- **Pipeline:** Contextual compression is typically the last pre-generation step after retrieval and re-ranking.

Combining Patterns: High-performing production RAG pipelines commonly stack: (1) Multi-query generation $\rightarrow$ (2) Hybrid retrieval $\rightarrow$ (3) Cross-encoder re-ranking $\rightarrow$ (4) Contextual compression. Each step incrementally improves answer quality at increasing latency cost. Profile before enabling all layers.

6 RAG Pipeline Architecture

A production RAG system encompasses two distinct pipelines: an offline indexing pipeline and an online query pipeline. Treating them as separate deployable components enables independent scaling and iteration.

6.1 Offline Indexing Pipeline

The indexing pipeline transforms raw documents into searchable vector representations:

1. **Document Ingestion:** Load from sources (S3, SharePoint, databases, web crawl). Support PDF (PyMuPDF, Unstructured.io), DOCX, HTML, Markdown, CSV.
2. **Preprocessing:** Normalise encoding (UTF-8), strip boilerplate (headers/footers), extract structured metadata (title, author, date, section).
3. **Chunking:** Apply strategy from §2. Attach metadata to each chunk (source file, chunk index, page number, section heading).
4. **Embedding:** Batch-encode chunks. Process in batches of 256–512 for API efficiency.
5. **Vector Store Upsert:** Write vectors + metadata to the vector DB. Use idempotent upsert (hash content as ID) to support incremental re-indexing.

6.2 Online Query Pipeline

The query pipeline handles real-time user requests:

1. **Query Pre-processing:** Safety checks (PII detection, prompt injection guard), language detection, query transformation (§5.1).
2. **Embedding:** Embed the (transformed) query using the same model as indexing. Target $< 50\text{ms}$ for embedding API calls.
3. **Retrieval:** Execute ANN search + optional BM25. Apply metadata filters (see §6.3). Retrieve top-50 to top-100 candidates.
4. **Re-ranking:** Apply cross-encoder or MMR to produce top-5 to top-10 final chunks.
5. **Contextual Compression:** Optionally extract relevant sub-spans.
6. **Prompt Assembly:** Inject retrieved context + conversation history + system prompt into the LLM template.
7. **Generation:** Call LLM API. Stream tokens to user if latency is critical.
8. **Post-processing:** Citation extraction, groundedness check, PII redaction in output.

6.3 Metadata Filtering

Metadata filters narrow the retrieval scope before ANN search, dramatically improving precision for structured corpora:

Filter Type	Example	Use Case
Date range	<code>created_at >= 2024-01-01</code>	Recency-sensitive queries

Filter Type	Example	Use Case
Source/domain	source = "legal-contracts"	Domain-specific retrieval
Document type	doc_type IN ["policy", "faq"]	Content-type filtering
User/tenant ID	tenant_id = {user.org}	Multi-tenant access control
Language	lang = "en"	Multilingual corpora

6.4 Vector Database Selection

Database	Scale	Metadata Filtering	Hybrid Search	Deployment
Pinecone	Billions	Rich	Yes (sparse-dense)	Managed cloud
Weaviate	100M+	GraphQL	Yes (BM25+vector)	Cloud / self-host
Qdrant	100M+	Payload filters	Yes	Cloud / self-host
Chroma	1M	Basic	No	Embedded / server
pgvector	10M	Full SQL	Via extension	PostgreSQL

6.5 Latency Budget

For an end-to-end latency target of 3 seconds (P95), a typical budget allocation:

- Query embedding: 50ms
- ANN retrieval (HNSW): 5–15ms
- BM25 retrieval: 10–30ms
- Cross-encoder re-ranking (top-50): 80–150ms
- LLM generation (first token, GPT-4o): 500–800ms
- LLM streaming (full response): 1500–2500ms

Performance Trap: Enabling contextual compression with an LLM extractor adds 300–600ms per retrieved chunk. For a top-5 retrieval set, this can add 1.5–3 seconds — potentially doubling total latency. Use the embeddings filter variant (§5.5) in latency-sensitive paths.

7 Evaluation Metrics and Frameworks

RAG evaluation is fundamentally a dual problem: evaluating retrieval quality (did we fetch the right chunks?) and generation quality (is the answer correct, faithful, and complete?). These require different metrics.

7.1 Retrieval Metrics

Precision@K

Fraction of retrieved chunks that are relevant:

$$\text{Precision@K} = |\{\text{relevant docs in top-K}\}| / K$$

Penalises noise in the retrieved set. Critical when context window is limited.

Recall@K

Fraction of all relevant chunks that appear in the top-K:

$$\text{Recall@K} = |\{\text{relevant docs in top-K}\}| / |\{\text{total relevant docs}\}|$$

Critical for completeness — missing a key supporting fact leads to incomplete answers.

Mean Reciprocal Rank (MRR)

Average of the reciprocal rank of the first relevant result across queries:

$$\text{MRR} = (1/|Q|) \cdot \sum_q 1 / \text{rank}_q(\text{first relevant})$$

Standard benchmark: MRR@10 on MS MARCO. State-of-the-art dense retrieval: MRR@10 ≈ 0.37–0.41.

Normalised Discounted Cumulative Gain (NDCG)

Considers both the relevance of retrieved documents and their rank position, supporting graded relevance judgements:

$$\text{NDCG@K} = \text{DCG@K} / \text{IDCG@K} \text{ where } \text{DCG@K} = \sum_i (2^{\text{rel}_i} - 1) / \log_2(i+1)$$

NDCG@10 is the primary metric for MS MARCO and BEIR benchmark evaluations. State-of-the-art hybrid retrieval: NDCG@10 ≈ 0.72–0.74 on MS MARCO.

7.2 Generation Metrics

Faithfulness

Measures whether every claim in the generated answer can be attributed to the retrieved context — i.e., the absence of hallucination grounded in retrieved chunks:

$$\text{Faithfulness} = |\{\text{claims supported by context}\}| / |\{\text{total claims in answer}\}|$$

Evaluated using an LLM-as-judge that decomposes the answer into atomic claims and verifies each against the source context. High-quality RAG systems target faithfulness > 0.90.

Answer Relevance

Measures whether the answer addresses the user's question (regardless of factual grounding). Computed by generating synthetic questions from the answer and measuring cosine similarity to the original query. Target: > 0.85.

Context Precision & Recall (RAGAS)

Context Precision assesses whether retrieved chunks contain the ground-truth answer. Context Recall measures what fraction of ground-truth answer statements can be attributed to the retrieved context.

7.3 RAGAS Framework

RAGAS (Retrieval Augmented Generation Assessment) is an open-source evaluation framework that provides reference-free, LLM-based scoring across four metrics:

Metric	Input Required	What It Measures	Target Score
Faithfulness	Q, Answer, Context	Claim-level hallucination rate	>0.90
Answer Relevance	Q, Answer	Query-answer alignment	>0.85
Context Precision	Q, Context, Ground Truth	Signal-to-noise in retrieved context	>0.80
Context Recall	Context, Ground Truth	Coverage of ground truth in context	>0.85

RAGAS uses GPT-4 or GPT-3.5-turbo as evaluator by default, consuming ~1000–2000 tokens per QA pair. For large test sets, use gpt-3.5-turbo with sampling to reduce evaluation cost by ~10×.

7.4 Evaluation Dataset Construction

- **Synthetic QA generation:** Use RAGAS TestsetGenerator to automatically generate question-answer pairs from the corpus. Generates simple, multi-context, and reasoning question types.
- **Human annotation:** Gold standard; expensive. Use for final acceptance testing (100–500 QA pairs per domain).
- **Production data:** Mine user queries + feedback labels (thumbs up/down) from production logs. Most representative of real usage.

7.5 Retrieval vs. Generation Decomposition

A key diagnostic: if faithfulness is high but answer relevance is low, the retrieval is returning irrelevant chunks (retrieval problem). If faithfulness is low but context recall is high, the LLM is generating unsupported claims (generation problem). This decomposition guides where to focus optimisation effort.

8 Common Pitfalls and Best Practices

Teams building RAG systems encounter a consistent set of failure patterns. Recognising these early prevents costly architectural rework.

8.1 Common Pitfalls

Pitfall 1: Ignoring Chunk Boundaries

Splitting mid-sentence or mid-table destroys semantic coherence. A chunk containing "...the result was 42.7%" with no referent context is useless for retrieval. **Fix:** Always inspect 10–20 sample chunks manually before committing to a chunking strategy.

Pitfall 2: Using the Wrong k

Retrieving too few chunks (k=3) risks missing supporting evidence for complex queries. Too many (k=20) floods the context window with noise, degrading generation quality. **Fix:** Start with k=5–8; tune on a held-out evaluation set measuring faithfulness and context precision jointly.

Pitfall 3: Embedding Model Mismatch

Indexing with one embedding model and querying with another produces incompatible vector spaces. **Fix:** Record the embedding model name and version in the vector store metadata; enforce version checks at query time.

Pitfall 4: Neglecting Metadata

Raw embedding search without metadata filters returns topically similar but contextually irrelevant documents (e.g., returning 2019 policy documents for a query about 2024 regulations). **Fix:** Extract and index date, source, document type, and access-level metadata at ingestion time.

Pitfall 5: No Evaluation Loop

Deploying RAG without automated evaluation means regressions go undetected. **Fix:** Integrate RAGAS or an equivalent into CI/CD. Run evaluation on every prompt template or chunking strategy change.

Pitfall 6: Lost-in-the-Middle Effect

Research (Liu et al., 2023) shows LLMs preferentially attend to content at the beginning and end of long context windows, ignoring middle content. **Fix:** Place the most relevant chunks first and last; apply contextual compression to reduce total context length.

8.2 Best Practices Checklist

Category	Best Practice	Priority
Chunking	Manually inspect 20+ sample chunks before deployment	HIGH
Chunking	Include chunk overlap (10–15%) to bridge boundary splits	HIGH
Embeddings	Pin embedding model version; re-index on model updates	HIGH
Retrieval	Use hybrid search (dense + BM25) for production systems	HIGH
Retrieval	Apply metadata filters to narrow the search scope	HIGH
Re-ranking	Deploy a cross-encoder re-ranker for precision-critical use cases	MED
Evaluation	Maintain a 200+ QA test set; run RAGAS in CI/CD	HIGH
Evaluation	Decompose failures into retrieval vs. generation problems	HIGH
Prompting	Instruct the LLM to answer only from provided context	HIGH
Prompting	Place most relevant chunks first in the context window	MED
Operations	Log retrieved chunk IDs and scores for every request	HIGH
Operations	Implement incremental re-indexing (hash-based upsert)	MED
Security	Apply tenant-level metadata filters to enforce access control	HIGH
Security	Detect and block prompt injection in user queries	HIGH

8.3 Scaling Considerations

- **Index size:** HNSW memory usage grows linearly with corpus size. For >10M vectors, switch to IVF-PQ or use disk-based ANN (DiskANN).
- **Throughput:** Embedding APIs typically rate-limit at 500–3000 RPM. Implement exponential backoff and request queuing for high-throughput indexing jobs.
- **Cost optimisation:** Cache embeddings for repeated queries (Redis TTL cache). Cache LLM responses for identical (query, context) pairs. A 90-day rolling cache can reduce API costs by 30–50% for stable corpora.
- **Corpus maintenance:** Stale documents degrade answer quality. Implement document TTL policies and re-indexing triggers on source system updates (webhooks, S3 event notifications).

8.4 Summary: RAG Maturity Model

Level	Characteristics	Typical RAGAS Score
L1 — Naive	Fixed-size chunks, dense-only, no re-ranking	0.55–0.65
L2 — Structured	Sentence/recursive chunking, hybrid search, metadata filters	0.65–0.75
L3 — Advanced	Semantic chunking, cross-encoder re-ranking, query transformation	0.75–0.85
L4 — Production	Parent-child, contextual compression, automated eval, caching	0.85–0.92